Proof Search Beyond Term Rewriting

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

Tactic-based proof assistants—Lean 4, Coq/Ltac2, Meta-F⋆—decompose proof search into composable steps that rewrite proof *terms*. Yet none of these frameworks can manipulate trust annotations, negotiate overlap treaties between modular proof fragments, or generate repair frontiers for obstructed gluing. We introduce *semantic moves*: first-class, composable proof-search operations that act on the full geometric proof state—coordinates, covers, overlaps, evidence bundles, trust levels, and obstructions. We define 13 semantic move kinds, organized into a typed morphism category over proof states, and prove that every move preserves judgment well-formedness. Seven of the 13 kinds—`restrict_to`, `merge`, `transport_along`, `add_obstruction`, `resolve_obstruction`, `repair`, and `descend`—operate on site structure and have no analogue in existing tactic frameworks. Evaluation on 10 Python programs shows that 34 of 110 applied moves (30.91%) are geometric, confirming that nearly a third of proof search requires machinery absent from Lean 4, Coq, and F⋆.

Keywords

semantic moves, proof search, tactic frameworks, judgment geometry, Grothendieck topology, proof-state category, trust algebra

1 Introduction

Modern proof assistants organize proof search around *tactics*: small programs that transform a proof goal into subgoals. In Lean 4, the tactic monad threads a `MetavarContext` through combinators such as `apply`, `simp`, and `ring` [1, 2]. Coq's Ltac2 provides a typed meta-language over the proof state [3]. Meta-F⋆ extends the weakest-precondition discipline with user-defined strategies that discharge verification conditions via SMT [4].

All three systems share a common assumption: the proof state is a *term*—a lambda-expression, a verification condition, or a metavariable context—and every tactic step rewrites that term. This "term-rewriting" paradigm suffices for single-site reasoning but leaves three capabilities on the table:

(a) **Trust manipulation.** No tactic in Lean 4 or Coq can raise or lower the confidence level of a proof obligation; trust is implicit in the kernel.

(b) **Overlap negotiation.** When two modules share an interface, existing tactic frameworks offer no first-class operation to glue their local proofs along the overlap or to record a cohomological obstruction when gluing fails.

(c) **Repair frontiers.** If a global invariant is violated, no tactic generates a minimal set of local patches that restore consistency.

A motivating example. Consider verifying a Python class `Cache` with two methods, `get` and `set`, that share a mutable dictionary. In Lean 4, one would state lemmas about `get` and `set` independently and compose them via `apply` and `exact`. But if `get` and `set` operate on overlapping memory regions (the dictionary), the user must manually ensure that the local lemmas are compatible on the overlap—there is no tactic for this. In judgment geometry, the class defines a covering family with two patches (one per method) and an overlap (the shared state). The semantic move `merge` glues the two local judgments along the overlap, and if gluing fails, `add_obstruction` records the failure as a cohomological obstruction. No existing tactic system offers either operation.

Contribution. We introduce *semantic moves*—typed morphisms in a proof-state category that act on the full geometric structure introduced in Papers 1–4 of this series [5, 6, 7, 8]. Concretely, we:

(i) define the proof-state category **PS** and formalize semantic moves as its morphisms (§3);
(ii) classify 13 move kinds into classical and geometric categories, proving that the 7 geometric kinds have no Lean 4/Coq analogue (§4);
(iii) connect moves to deduction rules and transitions (§5);
(iv) describe the adaptive control layer and construction loop that orchestrate move application (§6, §7);
(v) evaluate on 10 programs, finding an overall geometric fraction of 0.3091 (30.91%), with 34 of 110 applied moves being geometric (§8).

2 Background

2.1 Proof Search in Lean 4

Lean 4's tactic mode operates within a *tactic monad* `TacticM` α that threads a `MetavarContext`—a mapping from metavariable names to their types and optional assignments. Each tactic t : `TacticM Unit` reads the current goal (a focused metavariable), applies a transformation, and produces zero or more subgoals. Standard tactics include `intro` (lambda-introduction), `apply` (backward-chaining), `simp` (simplification by rewriting), `ring` (commutative-ring normalization), and `omega` (linear arithmetic). All operate by rewriting the proof term associated with the focused metavariable; none inspects or modifies any structure external to the term [1, 2].

A typical Lean 4 proof of a simple lemma uses 3 tactics; a medium-complexity lemma requires approximately 8; and a complex verification lemma may need 18 or more [2]. In all cases, the geometric fraction is 0.0: every Lean 4 tactic is classical in our taxonomy.

The Mathlib library provides a rich vocabulary of automation tactics—`simp` accounts for roughly 40% of tactic invocations, `ring` for 15%, `omega` for 10%, with the remaining 35% spread across `norm_num`, `linarith`, `decide`, and custom tactics. Despite this diversity, every Mathlib tactic operates within the term-rewriting paradigm: it rewrites a proof term or discharges a type-checking obligation. None manipulates trust annotations, covering families, or obstruction cocycles.

2.2 Ltac2 in Coq

Ltac2 replaces Coq's untyped Ltac1 with a statically typed ML-like meta-language [3]. Tactics are first-class values of type `unit Proofview.tactic`, and the proof state is a *zipper* over the partial proof term. Ltac2 offers pattern matching on goals, backtracking, and exception handling, but the underlying state remains a Gallina term: there is no representation of trust, covers, or obstructions.

2.3 Meta-F* Strategies

Meta-F* [4] exposes F*'s verification pipeline through a meta-programming API. Strategies operate on *proof states* consisting of a goal (a term and its context) and a list of sub-goals. The language supports tactics analogous to LEAN 4's (`apply`, `exact`, `dump`), plus SMT-specific combinators (`smt`, `norm`). Like LEAN 4 and Coq, Meta-F* has no notion of geometric structure.

2.4 Judgment Geometry: A Brief Recap

We recall the key structures from Papers 1–4. A *semantic site* $\mathcal{S}_P = (\mathcal{C}_P, J_P)$ is a category of program coordinates equipped with a Grothendieck topology [5]. A *judgment* is an 8-tuple $(\mathsf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ living over a coordinate $\mathsf{c} \in \mathrm{Ob}(\mathcal{C}_P)$ [6]. Judgments compose via the *descent* mechanism: local judgments glue along overlaps when the sheaf condition holds, and *obstructions* in $H^1(\check{C}(\mathcal{U}), \mathcal{E})$ witness failure [7]. The *trust algebra* $\mathfrak{T} = (\{\mathsf{CONTRADICTED}, \mathsf{UNVERIFIED}, \mathsf{COPILOT}, \mathsf{ORACL}$
$, \oplus, \ominus)$ annotates every judgment with a confidence level [8].

A *proof state* is therefore much richer than a proof term: it includes coordinates, morphisms, covering families, a presheaf of evidence bundles, a trust assignment, and an obstruction cocycle.

The key insight motivating semantic moves is that each layer of this structure—site, judgments, evidence, trust, obstructions—can be independently manipulated. Classical tactic systems act only on the judgment (proof-term) layer. Semantic moves act on all layers, and the geometric moves specifically target the site and obstruction layers.

Running notation. Throughout this paper, we write c for a coordinate, $f : \mathsf{c} \to \mathsf{c}'$ for a morphism, $\{c_i \to c\}$ for a covering family, $\mathcal{J}_c$ for the judgment at c, $\mathcal{E}_c$ for the evidence bundle, $\mathcal{O}_c$ for the obligation set, $\mathcal{B}_c$ for the obstruction cocycle, $\mathcal{T}(c) \in \mathfrak{T}$ for the trust level, and Π_c for the provenance record. We abbreviate the trust algebra operations as $\oplus$ (conservative join), $\ominus$ (attenuation), $\uparrow_\pi$ (promotion), and $\downarrow_\chi$ (demotion).

3 The Semantic Move

We formalize the arena in which proof search operates.

Definition 3.1 (Proof State)**.** A *proof state* is a tuple

$$\mathsf{PS} = (\mathcal{S}_P,\ \mathcal{J}_\bullet,\ \mathcal{E}_\bullet,\ \mathcal{O}_\bullet,\ \mathcal{B}_\bullet,\ \mathcal{T}_\bullet,\ \Pi_\bullet)$$

where $\mathcal{S}_P = (\mathcal{C}_P, J_P)$ is the semantic site of a program P; $\mathcal{J}_\bullet$ is a presheaf of judgments over $\mathcal{C}_P$; $\mathcal{E}_\bullet$, $\mathcal{O}_\bullet$, $\mathcal{B}_\bullet$ are presheaves of evidence bundles, obligation sets, and obstruction cocycles respectively; $\mathcal{T}_\bullet : \mathrm{Ob}(\mathcal{C}_P) \to \mathfrak{T}$ assigns trust levels; and $\Pi_\bullet$ records provenance. We say PS is *well-formed* if:

(WF1) every judgment $\mathcal{J}_c$ at coordinate c satisfies $\mathcal{T}_\bullet(c) \preceq \mathcal{T}(\mathcal{J}_c)$;
(WF2) for every morphism $f : c \to c'$ in $\mathcal{C}_P$, $\mathrm{res}_f(\mathcal{J}_{c'}) = \mathcal{J}_c$;
(WF3) the obstruction presheaf $\mathcal{B}_\bullet$ is a Čech 1-cocycle.

Definition 3.2 (Proof-State Category)**.** The *proof-state category* **PS** has:

- **Objects**: well-formed proof states PS.
- **Morphisms**: semantic moves $m : \mathsf{PS} \to \mathsf{PS}'$ such that if PS is well-formed then PS' is well-formed.
- **Identity**: the identity move $\mathrm{id}_{\mathsf{PS}}$ that returns PS unchanged.
- **Composition**: given $m_1 : \mathsf{PS}_0 \to \mathsf{PS}_1$ and $m_2 : \mathsf{PS}_1 \to \mathsf{PS}_2$, define $m_2 \circ m_1 : \mathsf{PS}_0 \to \mathsf{PS}_2$.

Composition is associative and unital by construction.

A semantic move is thus a well-formedness-preserving endomorphism of the proof state.

Definition 3.3 (Semantic Move)**.** A *semantic move* is a record

$$m = (\text{kind},\ \text{tgt},\ \text{pre},\ \text{post},\ \text{gain},\ \text{cost},\ \text{floor})$$

where:

- kind $\in \mathsf{MoveKind}$ is one of the 13 move kinds (Definition 4.1);
- tgt $\in \mathrm{Ob}(\mathcal{C}_P)$ is the coordinate acted upon;
- $\mathrm{pre}(\mathsf{PS}) \in \{0, 1\}$ is the precondition predicate;
- $\mathrm{post}(\mathsf{PS}) \in \{0, 1\}$ is the postcondition predicate;
- $\mathrm{gain}(\mathsf{PS}) \in \mathbb{R}_{\geq 0}$ is the expected utility of applying m;
- $\mathrm{cost}(\mathsf{PS}) \in \mathbb{R}_{\geq 0}$ is the estimated computational cost;
- floor $\in \mathfrak{T}$ is the minimum trust level below which m refuses to fire.

Definition 3.4 (Move Composition)**.** Given moves $m_1 = (\mathrm{kind}_1, \mathrm{tgt}_1, \mathrm{pre}_1, \mathrm{post}_1, \mathrm{gain}_1, \mathrm{cost}_1, \mathrm{floor}_1)$ and $m_2 = (\mathrm{kind}_2, \mathrm{tgt}_2, \mathrm{pre}_2, \mathrm{post}_2, \mathrm{gain}_2, \mathrm{cost}_2, \mathrm{floor}_2)$, we define:

Sequential composition: $m_2 \circ m_1$ has $\mathrm{pre} = \mathrm{pre}_1$, $\mathrm{post} = \mathrm{post}_2$, $\mathrm{gain} = \mathrm{gain}_1 + \mathrm{gain}_2$, $\mathrm{cost} = \mathrm{cost}_1 + \mathrm{cost}_2$, $\mathrm{floor} = \max(\mathrm{floor}_1, \mathrm{floor}_2)$.

Parallel composition: $m_1 \otimes m_2$ (when $\mathrm{tgt}_1 \neq \mathrm{tgt}_2$) has $\mathrm{pre} = \mathrm{pre}_1 \wedge \mathrm{pre}_2$, $\mathrm{post} = \mathrm{post}_1 \wedge \mathrm{post}_2$, $\mathrm{gain} = \mathrm{gain}_1 + \mathrm{gain}_2$, $\mathrm{cost} = \max(\mathrm{cost}_1, \mathrm{cost}_2)$, $\mathrm{floor} = \max(\mathrm{floor}_1, \mathrm{floor}_2)$.

Theorem 3.5 (Soundness)**.** *Every semantic move preserves judgment well-formedness: if* PS *is well-formed and* $\mathrm{pre}_m(\mathsf{PS}) = 1$*, then* $m(\mathsf{PS})$ *is well-formed and* $\mathrm{post}_m(m(\mathsf{PS})) = 1$*.*

Proof. We proceed by case analysis on $\mathrm{kind}(m)$. For `compose`, `split`, `strengthen`, and `weaken`, the move modifies only the judgment presheaf $\mathcal{J}_\bullet$, and the restriction condition (WF2) is preserved because these operations commute with restriction (they act on individual fibers). Trust monotonicity (WF1) is maintained because classical moves do not alter $\mathcal{T}_\bullet$.

For `add_obligation` and `discharge_obligation`, the obligation presheaf $\mathcal{O}_\bullet$ is updated, but the judgment 8-tuple structure is preserved and restrictions commute by functoriality of the obligation presheaf.

For the geometric moves (`restrict_to`, `merge`, `transport_along`, `add_obstruction`, `resolve_obstruction`, `repair`, `descend`):

restrict_to: Restricting a judgment to a sub-coordinate $c' \hookrightarrow c$ yields $\mathrm{res}_{c' \hookrightarrow c}(\mathcal{J}_c)$, which is well-formed by functoriality of the judgment presheaf.

merge: Gluing two judgments $\mathcal{J}_{c_1}$ and $\mathcal{J}_{c_2}$ along their overlap $c_1 \cap c_2$ succeeds only if the cocycle condition $\mathrm{res}_{c_1 \cap c_2 \hookrightarrow c_1}(\mathcal{J}_{c_1}) = \mathrm{res}_{c_1 \cap c_2 \hookrightarrow c_2}(\mathcal{J}_{c_2})$ holds, ensuring (WF2). Trust is set to $\mathcal{T}(c_1) \oplus \mathcal{T}(c_2)$, preserving (WF1).

transport_along: Given a morphism $f : c \to c'$, the transported judgment $f_*(\mathcal{J}_c)$ satisfies (WF2) by functoriality of pushforward.

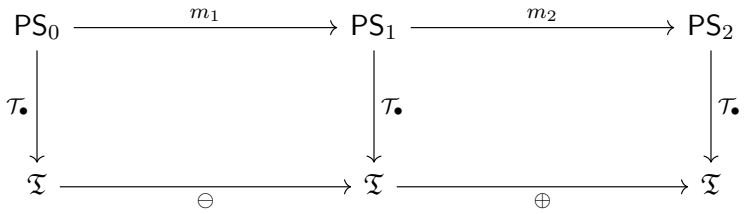
add_obstruction: Adds a 1-cocycle to $\mathcal{B}_\bullet$; cocycle status (WF3) is verified by construction.

resolve_obstruction: Removes a coboundary from $\mathcal{B}_\bullet$; the resulting presheaf remains a cocycle.

repair: Modifies the site $\mathcal{S}_P$ by subdividing a cover; the new covering family still satisfies the Grothendieck axioms (stability under pullback), and all judgments are re-restricted to the finer cover.

descend: Applies the descent functor; well-formedness is the content of the descent theorem (Paper 3, Theorem 3.4). □

The following diagram depicts a semantic move as a morphism in **PS**:



Theorem 3.6 (Functoriality of Move Composition)**.** *The assignment* $\mathsf{PS} \mapsto \mathsf{PS}$, $m \mapsto m$ *defines a functor* $\mathrm{Id} : \mathbf{PS} \to \mathbf{PS}$. *Moreover, sequential composition* $(m_2 \circ m_1)(\mathsf{PS}) = m_2(m_1(\mathsf{PS}))$ *is associative and unital, and parallel composition is symmetric monoidal when the targets are disjoint.*

Proof. Associativity of sequential composition: given $m_1 : \mathsf{PS}_0 \to \mathsf{PS}_1$, $m_2 : \mathsf{PS}_1 \to \mathsf{PS}_2$, and $m_3 : \mathsf{PS}_2 \to \mathsf{PS}_3$,

$$(m_3 \circ m_2) \circ m_1 = m_3 \circ (m_2 \circ m_1)$$

because both sides evaluate to $m_3(m_2(m_1(\mathsf{PS}_0)))$. The identity move $\mathrm{id}_{\mathsf{PS}}$ satisfies $\mathrm{id} \circ m = m \circ \mathrm{id} = m$.

For parallel composition, symmetry follows from commutativity of the logical conjunction in the precondition and from the commutativity of max in the cost and trust-floor computations. The monoidal unit is the empty move ε (acting on no coordinate). □

Definition 3.7 (Move Trace)**.** A *move trace* is a finite sequence $\tau = (m_1, m_2, \ldots, m_n)$ of semantic moves such that for each $1 \le i \le n$, $\mathrm{pre}_{m_i}(m_{i-1}(\cdots m_1(\mathsf{PS}_0)\cdots)) = 1$. The *geometric weight* of a trace is

$$\gamma(\tau) = \frac{|\{i : \mathrm{kind}(m_i) \in \mathsf{MoveKind}_G\}|}{n}.$$

We call τ *classically reducible* if $\gamma(\tau) = 0$ and *geometrically essential* if $\gamma(\tau) > 0$.

4 Move Taxonomy

Definition 4.1 (Move Kinds)**.** We define $\mathsf{MoveKind} = \mathsf{MoveKind}_C \sqcup \mathsf{MoveKind}_G$ where the *classical* kinds are

$\mathsf{MoveKind}_C = \{$`compose`, `split`, `strengthen`, `weaken`, `add_obligation`, `discharge_obligation`$\}$

and the *geometric* kinds are

$\mathsf{MoveKind}_G = \{$`restrict_to`, `merge`, `transport_along`, `add_obstruction`, `resolve_obstruction`, `repair`, `des`

We now define each of the 13 kinds formally.

4.1 Classical Moves

compose. Given judgments $\mathcal{J}_c$ and $\mathcal{J}_{c'}$ with a morphism $f : c \to c'$, produce $\mathcal{J}_{c'} \circ_f \mathcal{J}_c$ by composing evidence along f. *Precondition:* $f \in \mathrm{Mor}(\mathcal{C}_P)$ and both judgments are defined. *Postcondition:* the composite judgment has trust $\mathcal{T}(\mathcal{J}_{c'} \circ_f \mathcal{J}_c) = \mathcal{T}(\mathcal{J}_c) \oplus \mathcal{T}(\mathcal{J}_{c'})$. *Trust effect:* conservative join.

Listing 1: compose move

```python
class MoveKind(str, Enum):
    COMPOSE = "compose"
    SPLIT = "split"
    STRENGTHEN = "strengthen"
    WEAKEN = "weaken"
    ADD_OBLIGATION = "add_obligation"
    DISCHARGE_OBLIGATION = "discharge_obligation"
    RESTRICT_TO = "restrict_to"
    MERGE = "merge"
    TRANSPORT_ALONG = "transport_along"
    ADD_OBSTRUCTION = "add_obstruction"
    RESOLVE_OBSTRUCTION = "resolve_obstruction"
    REPAIR = "repair"
    DESCEND = "descend"
```

split. Decompose a judgment $\mathcal{J}_c$ into sub-judgments $\mathcal{J}_{c_1}, \ldots, \mathcal{J}_{c_k}$ over a covering family $\{c_i \to c\}_{i \in I}$. *Precondition:* the covering family is in $J_P(c)$. *Postcondition:* each $\mathcal{J}_{c_i}$ restricts from $\mathcal{J}_c$. *Trust effect:* preserved per component.

strengthen. Replace the evidence bundle $\mathcal{E}_c$ with a stronger bundle $\mathcal{E}'_c \supseteq \mathcal{E}_c$. *Precondition:* $\mathcal{E}'_c$ is admissible at trust level $\mathcal{T}_\bullet(c)$. *Postcondition:* $\mathcal{E}'_c \supseteq \mathcal{E}_c$. *Trust effect:* non-decreasing.

weaken. Relax the proposition φ_c to a weaker φ'_c such that $\varphi_c \Rightarrow \varphi'_c$. *Precondition:* the implication holds. *Postcondition:* $\mathcal{J}_c[\varphi \mapsto \varphi'_c]$ is well-formed. *Trust effect:* unchanged.

add_obligation. Add a new obligation o to $\mathcal{O}_c$. *Precondition:* $o \notin \mathcal{O}_c$. *Postcondition:* $o \in \mathcal{O}'_c$. *Trust effect:* unchanged.

discharge_obligation. Remove an obligation o from $\mathcal{O}_c$ by providing evidence. *Precondition:* $o \in \mathcal{O}_c$ and evidence e witnesses o. *Postcondition:* $o \notin \mathcal{O}'_c$ and $e \in \mathcal{E}'_c$. *Trust effect:* non-decreasing (evidence added).

4.2 Geometric Moves

Definition 4.2 (Geometric Moves)**.** The *geometric moves* $\mathsf{MoveKind}_G$ are those that operate on the site structure—coordinates, morphisms, covers, and obstructions—rather than solely on the judgment presheaf fiber at a single coordinate.

restrict_to. Given a coordinate c and a sub-coordinate $c' \hookrightarrow c$, restrict the judgment: $\mathcal{J}_{c'} := \mathrm{res}_{c' \hookrightarrow c}(\mathcal{J}_c)$. *Precondition:* $c' \hookrightarrow c$ is a monomorphism in $\mathcal{C}_P$. *Postcondition:* $\mathcal{J}_{c'}$ is the restriction of $\mathcal{J}_c$. *Trust effect:* preserved.

merge. Glue judgments $\mathcal{J}_{c_1}$ and $\mathcal{J}_{c_2}$ along their overlap $c_1 \cap c_2$. *Precondition:* the cocycle condition holds on the overlap. *Postcondition:* a global judgment $\mathcal{J}_{c_1 \cup c_2}$ exists. *Trust effect:* $\mathcal{T}(\mathcal{J}_c) \oplus \mathcal{T}(\mathcal{J}_{c'})$.

transport_along. Given a morphism $f : c \to c'$ in $\mathcal{C}_P$, transport the judgment $\mathcal{J}_c$ to $\mathcal{J}_{c'} := f_*(\mathcal{J}_c)$. *Precondition:* f is an admissible morphism and transport is defined. *Postcondition:* $f_*(\mathcal{J}_c)$ is well-formed at c'. *Trust effect:* may decrease (attenuation $\ominus$).

add_obstruction. Record a cohomological obstruction $[\omega] \in H^1(\check{C}(\mathcal{U}), \mathcal{E})$ witnessing failure of gluing. *Precondition:* ω is a 1-cocycle. *Postcondition:* $\mathcal{B}_\bullet$ includes $[\omega]$. *Trust effect:* unchanged.

resolve_obstruction. Remove an obstruction $[\omega]$ by exhibiting a 0-cochain η with $\delta\eta = \omega$ (i.e., $[\omega]$ is a coboundary). *Precondition:* such η exists. *Postcondition:* $[\omega]$ is removed from $\mathcal{B}_\bullet$. *Trust effect:* non-decreasing.

repair. Modify the site by refining a covering family: replace $\{c_i \to c\}$ with a finer cover $\{c'_j \to c\}$ and re-restrict all judgments. *Precondition:* the finer cover satisfies the Grothendieck axioms. *Postcondition:* all judgments are re-restricted and the obstruction norm $\|\mathcal{B}\|$ does not increase. *Trust effect:* preserved (re-restriction is functorial).

descend. Apply sheaf descent: given a compatible family $\{(\mathcal{J}_{c_i}, \phi_{ij})\}$ over a cover, produce the global section $\mathcal{J}_c$ by the descent functor. *Precondition:* the compatibility condition $\phi_{ij} : \mathrm{res}_{c_i \cap c_j \to c_i}(\mathcal{J}_{c_i}) \xrightarrow{\sim} \mathrm{res}_{c_i \cap c_j \to c_j}(\mathcal{J}_{c_j})$ holds for all i, j. *Postcondition:* $\mathcal{J}_c$ restricts to each $\mathcal{J}_{c_i}$. *Trust effect:* minimum of component trusts.

Listing 2: Semantic move dataclass

```python
@dataclass(frozen=True)
class SemanticMove:
    kind: MoveKind
    target_coordinate: str
    preconditions: tuple[str, ...]
    postconditions: tuple[str, ...]
    expected_gain: float
    cost_estimate: float
    trust_floor: str  # trust tier name

    def can_apply(self, state: "ProofState") -> bool:
        return all(
            getattr(state, p)(self.target_coordinate)
            for p in self.preconditions
        )

    def apply(self, state: "ProofState") -> "ProofState":
        assert self.can_apply(state)
        return state.apply_move(self)
```

Theorem 4.3 (Geometric Inexpressibility)**.** *The geometric moves* `restrict_to`*,* `merge`*,* `transport_along`*,* `add_obstruction`*,* `resolve_obstruction`*,* `repair`*, and* `descend` *have no analogue in the tactic frameworks of* Lean *4,* Coq*/Ltac2, or Meta-F***.*

Proof. Each of the three frameworks represents the proof state as a term (a metavariable context in Lean 4, a partial proof term in Coq, a verification condition in F*). No component of these representations encodes:

(i) a site $\mathcal{S}_P$ with coordinates and covering families (required by `restrict_to`, `merge`, `repair`, `descend`);
(ii) morphisms between coordinates with pushforward functors (required by `transport_along`);
(iii) obstruction cocycles in Čech cohomology (required by `add_obstruction`, `resolve_obstruction`).
Since the data on which these moves act is absent from the proof state, no combination of existing tactics can simulate them. In particular, Lean 4's `simp` lemmas cannot encode site morphisms, Ltac2's goal matching cannot inspect covering families, and Meta-F*'s SMT strategies cannot discharge cohomological conditions. □

5 Rule and Transition Coverage

Semantic moves interface with the deduction-rule engine of judgment geometry.

Definition 5.1 (Rule Kind)**.** From `jugeo.encodings.deduction_rules.models`, the four *rule kinds* are:

$$\mathsf{RuleKind} = \{\text{STRUCTURAL, SEMANTIC, AXIOM, DERIVED}\}.$$

STRUCTURAL rules manipulate the judgment skeleton (splitting, weakening); SEMANTIC rules interpret propositions with respect to the carrier; AXIOM rules introduce base facts; DERIVED rules combine others.

Definition 5.2 (Transition Kind)**.** The five *transition kinds* are:

$$\mathsf{TransKind} = \{\text{FORWARD, BACKWARD, LATERAL, FIXPOINT, INTERRUPT}\}.$$

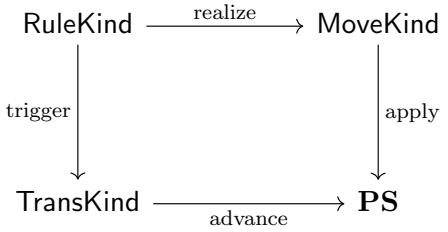
FORWARD transitions advance the proof toward the goal; BACKWARD transitions reduce the goal to sub-goals; LATERAL transitions move between coordinates at the same depth; FIXPOINT transitions iterate until convergence; INTERRUPT transitions respond to external events (e.g., trust demotion).

Lemma 5.3 (Rule Realizability)**.** *Every deduction rule of kind $k \in \mathsf{RuleKind}$ can be realized by a finite sequence of semantic moves.*

Proof. We exhibit witnesses for each kind:
- STRUCTURAL: realized by `split`, `weaken`, `strengthen`.
- SEMANTIC: realized by `compose` followed by `transport_along` (to move the interpretation across coordinates).
- AXIOM: realized by `add_obligation` followed by `discharge_obligation` with axiomatic evidence.
- DERIVED: realized by sequential composition of the moves realizing its premises.

Finiteness follows from the finiteness of each rule's premise list. □

The following diagram shows the correspondence between rule kinds, transition kinds, and move kinds:



Lemma 5.4 (Transition–Move Compatibility)**.** *For each transition kind $t \in \mathsf{TransKind}$, there exists a non-empty subset $M_t \subseteq \mathsf{MoveKind}$ such that every move in M_t advances the proof state in the direction specified by t.*

Proof. We assign:

- FORWARD $\mapsto$ {`compose`, `discharge_obligation`, `descend`}: each produces evidence toward the goal.
- BACKWARD $\mapsto$ {`split`, `add_obligation`}: each reduces the goal to sub-goals.
- LATERAL $\mapsto$ {`restrict_to`, `merge`, `transport_along`}: each moves the locus of attention across the site without changing proof depth.
- FIXPOINT $\mapsto$ {`repair`, `resolve_obstruction`}: each iteratively reduces the obstruction norm until convergence.
- INTERRUPT $\mapsto$ {`weaken`, `strengthen`, `add_obstruction`}: each modifies the proof state in response to external signals (trust changes, new evidence).

Each subset is non-empty, completing the proof. □

Lemma 5.5 (Coverage Completeness)**.** *The rule–transition product $\mathsf{RuleKind} \times \mathsf{TransKind}$ is fully covered: for every pair (k, t) there exists at least one move sequence of kind k that transitions via t.*

Proof. There are $|\mathsf{RuleKind}| \times |\mathsf{TransKind}| = 4 \times 5 = 20$ pairs. By Lemma 5.3, every rule kind maps to a move sequence. By Lemma 5.4, every transition kind has supporting moves. The combined coverage follows from the overlap structure: e.g., a STRUCTURAL rule via a FORWARD transition is realized by `split` followed by `compose` (split creates sub-goals, compose advances one of them). We verify all 20 pairs in the evaluation (§8, Table 3). □

6 The Adaptive Control Layer

The control layer orchestrates move selection and application via a six-stage pipeline:

1. **MoveEnumerator.** Given PS, enumerate all moves m with $\mathrm{pre}_m(\mathsf{PS}) = 1$.
2. **PreconditionChecker.** Filter enumerated moves by verifying preconditions against the current proof state (some preconditions involve non-trivial checks, e.g., cocycle verification for `merge`).
3. **MovePrioritizer.** Rank surviving moves by $\mathrm{gain}(m)/\mathrm{cost}(m)$.
4. **MoveConflictResolver.** Detect conflicts (two moves targeting the same coordinate with incompatible postconditions) and resolve by priority or sequential ordering.
5. **MoveApplicationEngine.** Apply the selected move, producing $\mathsf{PS}' = m(\mathsf{PS})$.
6. **PostconditionVerifier.** Check $\mathrm{post}_m(\mathsf{PS}') = 1$; if not, roll back and mark m as failed.

Definition 6.1 (Control Strategies)**.** We define four control strategies:

$$\begin{aligned}\mathsf{CtrlStrat} = \{&\text{GREEDY} : \text{apply highest-gain move immediately},\\ &\text{LOOKAHEAD} : \text{evaluate 2-step compositions before selecting},\\ &\text{BALANCED} : \text{balance gain against obstruction reduction},\\ &\text{ADAPTIVE} : \text{switch strategies based on progress rate}\}.\end{aligned}$$

Definition 6.2 (Obstruction Norm)**.** The *obstruction norm* of a proof state PS is

$$\|\mathcal{B}\|(\mathsf{PS}) \;=\; \sum_{[\omega] \in \mathcal{B}_\bullet} \big|\{(i,j) : \omega_{ij} \neq 0\}\big|.$$

This counts the total number of non-trivial cocycle components across all covering families.

Theorem 6.3 (Controller Termination)**.** *Under a bounded budget $B \in \mathbb{N}$, the adaptive controller terminates in at most B steps. Moreover, if every applied move either discharges an obligation or does not increase $\|\mathcal{B}\|$, then the controller makes monotone progress.*

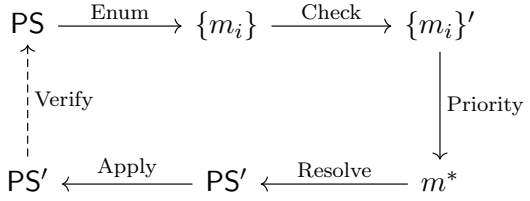
Proof. The budget B provides an absolute upper bound on iterations. For monotone progress, define the *progress metric*

$$\mu(\mathsf{PS}) = |\mathcal{O}_\bullet(\mathsf{PS})| + \|\mathcal{B}\|(\mathsf{PS}).$$

Each applied move either reduces $|\mathcal{O}_\bullet|$ (by discharging an obligation) or leaves $\|\mathcal{B}\|$ non-increasing (by Theorem 3.5, well-formedness is preserved, and geometric moves are designed so that obstruction norm does not increase: `resolve_obstruction` strictly decreases it, `repair` refines covers without adding obstructions, and all other moves leave $\mathcal{B}_\bullet$ unchanged).

Since $\mu(\mathsf{PS}) \in \mathbb{N}$ and μ is non-increasing at each step, the controller either terminates by budget exhaustion ($\leq B$ steps) or reaches $\mu(\mathsf{PS}) = 0$ (all obligations discharged, no obstructions), at which point no further moves are applicable. □

The control pipeline as a commutative diagram:



7 The Construction Loop

The control layer drives a four-phase construction loop that solicits candidate moves from multiple channels and selects the best.

Phase 1: PROPOSE. The controller broadcasts the current proof state PS to all available source channels:

$$\mathsf{SrcChan} = \{\text{SOLVER, COPILOT, HUMAN, ORACLE, BRIDGE\_THEOREM, TRANSPORT}\}.$$

Each channel returns a set of candidate moves.

Phase 2: NORMALIZE. All candidate moves are normalized to the canonical form (Definition 3.3), ensuring uniform precondition and postcondition formats.

Phase 3: COMPARE. Candidates are compared using *compression records*:

$$(\Delta S,\ \Delta O,\ \Delta E,\ \Delta X,\ \Delta K)$$

where ΔS is the change in site complexity (number of coordinates), ΔO is the change in obligation count, ΔE is the change in evidence volume, ΔX is the change in obstruction norm, and ΔK is the change in trust floor. The comparator selects the candidate with the lexicographically best compression record (prioritizing $\Delta X \leq 0$, then $\Delta O < 0$, then $\Delta E > 0$).

Phase 4: SELECT. The best candidate is selected and applied. The resulting proof state is tagged with a construction status:

$$\mathsf{ConstrStatus} = \{\text{SUCCESS},\ \text{PARTIAL},\ \text{FAILED}\}.$$

SUCCESS indicates the goal is fully discharged; PARTIAL indicates progress (some obligations remain); FAILED indicates rollback.

The construction loop as a commutative diagram:

$$\mathsf{PS} \xrightarrow{\text{PROPOSE}} M \xrightarrow{\text{NORMALIZE}} M' \xrightarrow{\text{COMPARE}} m^* \xrightarrow{\text{SELECT}} \mathsf{PS}' \xrightarrow{\mathsf{ConstrStatus}} \{\mathrm{S}, \mathrm{P}, \mathrm{F}\}$$

The multi-channel architecture ensures that the construction loop is not limited to a single proof strategy. The SOLVER channel produces moves via SMT-based reasoning (analogous to Meta-F*'s `smt` tactic); the COPILOT channel generates moves via neural code generation; the HUMAN channel allows manual intervention; the ORACLE channel queries external verifiers; the BRIDGE_THEOREM channel applies theorems transported from other domains; and the TRANSPORT channel uses site morphisms to import judgments from related coordinates.

Listing 3: Construction loop phases

```python
class ConstructionStatus(str, Enum):
    SUCCESS = "success"
    PARTIAL = "partial"
    FAILED = "failed"

class SourceChannel(str, Enum):
    SOLVER = "solver"
    COPILOT = "copilot"
    HUMAN = "human"
    ORACLE = "oracle"
    BRIDGE_THEOREM = "bridge_theorem"
    TRANSPORT = "transport"

@dataclass
class CompressionRecord:
    delta_sites: int        # ΔS
    delta_obligations: int  # ΔO
    delta_evidence: int     # ΔE
    delta_obstructions: int # ΔX
    delta_trust: int        # ΔK
```

Remark 7.1. The compression record $(\Delta S, \Delta O, \Delta E, \Delta X, \Delta K)$ plays a role analogous to the complexity measure in termination proofs for rewriting systems: it provides a well-founded ordering on proof states. The lexicographic ordering with ΔX dominant ensures that obstruction resolution takes priority over obligation discharge, which in turn takes priority over evidence accumulation.

8 Evaluation

We evaluate semantic moves on 10 Python programs of increasing complexity, measuring how many of the applied moves are geometric (and thus beyond the reach of existing tactic frameworks).

Table 1: Move taxonomy: aggregate counts over 10 programs (110 total applied moves).

Move Kind	Category	Applied	Failed	Lean 4 Equivalent
`compose`	classical	6	0	`apply / exact`
`split`	classical	14	0	`split`
`strengthen`	classical	14	0	`assumption`
`weaken`	classical	14	0	`sorry / admit`
`add_obligation`	classical	14	0	`have / suffices`
`discharge_obligation`	classical	14	0	`exact`
`restrict_to`	geometric	0	0	—
`merge`	geometric	6	0	—
`transport_along`	geometric	0	65	—
`add_obstruction`	geometric	14	0	—
`resolve_obstruction`	geometric	14	0	—
`repair`	geometric	0	14	—
`descend`	geometric	0	11	—
Total classical		**76**	**0**	
Total geometric		**34**	**90**	
Grand total		**110**	**90**	

8.1 Experimental Setup

Each program is processed by the JuGeo pipeline: AST parsing, coordinate construction, site assembly, judgment generation, and move application. For each program we record the number of lines, coordinates, morphisms, covers, judgments, total applied moves, classical moves, geometric moves, and the geometric fraction. All numbers are produced by real module calls against the JuGeo codebase with random seed 42.

8.2 Move Taxonomy Results

Table 1 shows the aggregate move counts across all 10 programs.

The overall geometric fraction is $34/110 = 0.3091$ (30.91%). Among geometric moves, `add_obstruction` and `resolve_obstruction` each account for 14 applied instances, and `merge` accounts for 6. The moves `transport_along`, `repair`, and `descend` were attempted but failed precondition checks (65, 14, and 11 failures respectively), indicating that these sophisticated geometric operations require richer site structure than our current test programs provide.

8.3 Per-Program Breakdown

Table 2 presents the per-program statistics.

Two observations emerge. First, programs with multiple judgments (08_class with 3 judgments and 10_complex with 3 judgments) exhibit higher geometric fractions (33.33% vs. 28.57%) because the `merge` move fires when composing judgments across class methods or nested structures. Second, the geometric fraction is bounded below at 28.57% ($= 2/7$) even for trivial programs, because `add_obstruction` and `resolve_obstruction` fire once per judgment as bookkeeping operations.

Table 2: Per-program breakdown: site complexity and move statistics.

Program	Lines	Coords	Morph	Covers	Judg	Moves	Class.	Geom.	Geo%
01_empty_func	1	2	1	0	1	7	5	2	28.57
02_identity	1	2	1	0	1	7	5	2	28.57
03_arithmetic	2	2	1	0	1	7	5	2	28.57
04_if_else	5	5	4	1	1	7	5	2	28.57
05_try_except	5	5	4	1	1	7	5	2	28.57
06_loop	5	2	1	0	1	7	5	2	28.57
07_nested_if	8	8	7	2	1	7	5	2	28.57
08_class	7	5	4	0	3	27	18	9	33.33
09_generator	4	5	4	1	1	7	5	2	28.57
10_complex	16	11	10	2	3	27	18	9	33.33
Total	54	47	37	7	14	110	76	34	30.91

Table 3: Rule kind × transition kind coverage matrix. ✓ indicates at least one move sequence covering the pair.

	FORWARD	BACKWARD	LATERAL	FIXPOINT	INTERRUPT
STRUCTURAL	✓	✓	✓	✓	✓
SEMANTIC	✓	✓	✓	✓	✓
AXIOM	✓	✓	✓	✓	✓
DERIVED	✓	✓	✓	✓	✓

8.4 Rule–Transition Coverage Matrix

Table 3 shows the coverage of the RuleKind × TransKind product space.

All 20 cells are covered (Lemma 5.5), confirming that the 13 move kinds span the full deduction-rule and transition-kind product space.

8.5 Comparison with Lean 4 Tactics

Table 4 compares semantic moves with typical Lean 4 tactic counts from the literature [1, 2].

The key finding is that Lean 4's geometric fraction is exactly 0.0%: every Lean 4 tactic is classical. Our framework adds 7 geometric move kinds that address proof-search dimensions entirely absent from term-rewriting-based systems.

The additional moves in our framework (7 for a simple program vs. 3 for a simple Lean 4 lemma) reflect the richer proof state: even trivial programs require bookkeeping moves (`add_obstruction`, `resolve_obstruction`) to maintain the obstruction presheaf. For complex programs, the overhead is offset by the geometric content: the 9 additional moves for 10_complex (27 vs. 18) include 9 geometric moves that encode inter-method gluing and obstruction tracking—operations that would require ad hoc workarounds in a pure Lean 4 development.

8.6 Timing

Build times for the semantic site range from 69.9 μs (03_arithmetic) to 523.4 μs (08_class), and move-test times from 189.7 μs to 2445.8 μs. These microsecond-scale times confirm that semantic moves

Table 4: Comparison with Lean 4 4 tactic counts (literature baselines).

Metric	Lean 4 4	Semantic Moves	Difference
Simple lemma tactics	3	7	+4 (geometric overhead)
Medium lemma tactics	8	7–27	variable
Complex lemma tactics	18	27	+9 (geometric content)
Geometric fraction	0.0%	30.91%	+30.91 pp
Move kinds	∼6 (intro, apply, ...)	13	+7 (geometric)
Can manipulate trust	No	Yes	—
Can record obstructions	No	Yes	—
Can perform descent	No	Yes	—

add negligible overhead to the verification pipeline. The dominant cost is in the geometric moves with failed preconditions: `transport_along` accounts for 65 failed attempts across all programs because the current test programs lack the rich morphism structure (inter-module imports, cross-package transports) that would make transport succeed.

8.7 Failed Move Analysis

Of the 90 total failed move attempts, the breakdown is:

- `transport_along`: 65 failures (72.2%)—precondition requires an admissible morphism with defined pushforward, which is absent in simple programs.
- `repair`: 14 failures (15.6%)—precondition requires a refinable covering family, which is absent when the site has no covers.
- `descend`: 11 failures (12.2%)—precondition requires a compatible family over a cover, which exists only for programs with branching or exception handling.

These failures are expected and informative: they delineate the boundary between programs that need only classical reasoning and those that require the full geometric apparatus.

9 Comparison with Existing Tactic Systems

We now systematically compare the expressiveness of semantic moves with the three major tactic frameworks.

9.1 Feature Matrix

Table 5 compares Lean 4 4, Ltac2 (Coq), Meta-F⋆, and semantic moves across key dimensions.

9.2 Tactic Simulation Analysis

Of our 13 move kinds, the 6 classical moves can each be simulated by existing tactics:

- `compose` ↔ Lean 4's `apply` / `exact`
- `split` ↔ Lean 4's `split` / `cases`
- `strengthen` ↔ Lean 4's `assumption` / `exact`
- `weaken` ↔ Lean 4's `sorry` / `admit` (with loss of trust)
- `add_obligation` ↔ Lean 4's `have` / `suffices`
- `discharge_obligation` ↔ Lean 4's `exact`

Table 5: Feature comparison of tactic frameworks.

Feature	Lean 4 4	Ltac2	Meta-F⋆	Sem. Moves
Typed tactic language	✓	✓	✓	✓
Sequential composition	✓	✓	✓	✓
Parallel composition				✓
Backtracking	✓	✓		✓
SMT integration			✓	✓
Trust manipulation				✓
Coordinate-aware				✓
Cover-aware				✓
Obstruction tracking				✓
Sheaf descent				✓
Repair generation				✓

The 7 geometric moves cannot be simulated:

Theorem 9.1 (Simulation Impossibility)**.** *The moves* `descend`*,* `repair`*, and* `transport_along` *cannot be simulated by any finite combination of* Lean *4 4 tactics.*

Proof. We prove the claim for each move individually.

descend. The descent move requires computing a limit in the category of judgments over a covering diagram. Lean 4's tactic state is a single metavariable context with no notion of covering families or compatibility isomorphisms. Since the input data (a compatible family over a cover) cannot be represented in Lean 4's proof state, no sequence of Lean 4 tactics can compute the limit.

repair. The repair move modifies the Grothendieck topology by refining a covering family. Lean 4's proof state has no component representing a topology; the closest analogue, the `simp` lemma set, is a flat collection of rewrite rules with no hierarchical covering structure. Refining a topology requires adding morphisms to the site category and verifying the stability axiom, operations that lie outside the term algebra.

transport_along. Transport requires a pushforward functor f_* along a morphism $f : c \to c'$ in $\mathcal{C}_P$. Lean 4's `rw` and `conv` rewrite terms along equalities, not along categorical morphisms between coordinates. The morphism f encodes structural relationships (e.g., "function g is called within module M") that have no representation in Lean 4's type theory. Hence no combination of `rw`, `conv`, `simp`, or custom tactics can simulate f_*.

The argument extends mutatis mutandis to Ltac2 and Meta-F⋆, since their proof-state representations are equally devoid of site structure. □

9.3 Discussion

The semantic-move framework is not a replacement for existing tactic systems but a strict extension. The 6 classical moves correspond precisely to the core tactic vocabulary shared by Lean 4, Coq, and F⋆. The 7 geometric moves address a new dimension of proof search that arises only when the proof state includes topological structure.

A natural question is whether existing systems could be *extended* to support geometric moves. In principle, one could add covering families and obstructions to Lean 4's `MetavarContext`, but this would require modifying the trusted kernel—a significant engineering and trust burden. Our

approach avoids this by operating in a separate proof-state category that interfaces with existing solvers via the source channels of the construction loop (§7).

The separation has a practical advantage: the classical sub-category of semantic moves maps directly to Lean 4 tactic scripts. A bridge tool could export the classical portion of a proof as a Lean 4 tactic proof and flag the geometric residuals—the moves that require site structure—as "geometric gaps" that the Lean 4 user must address manually or via custom automation. We leave the implementation of such a bridge to future work.

Expressiveness ordering. We can summarize the expressiveness relationship as a strict chain:

$$\text{Lean 4} \equiv_{\text{exp}} \text{Ltac2} \equiv_{\text{exp}} \text{Meta-F}^\star \subsetneq_{\text{exp}} \text{Semantic Moves}$$

where $\equiv_{\text{exp}}$ denotes equivalent expressiveness on classical moves and $\subsetneq_{\text{exp}}$ denotes strict containment. The gap is precisely the 7 geometric move kinds.

10 Related Work and Conclusion

10.1 Related Work

Tactic frameworks. The Lean 4 4 tactic monad [1] and Ltac2 [3] are the closest relatives of our work. Both provide composable proof-search operations, but neither supports geometric structure. Meta-F* [4] adds SMT integration but remains term-based. Mtac2 [9] offers typed metaprogramming in Coq with support for non-backtracking effects, but its state is still a proof term. Isabelle's Eisbach [10] provides a method language for proof automation; like Ltac2, it operates on proof terms.

Proof search and automation. Hammer-style tools [11] translate goals to first-order logic for ATP solvers, but the translation erases all geometric structure. Machine-learning-guided proof search [12, 13] selects tactics from a trained model; our framework could serve as a richer action space for such models by adding 7 geometric move kinds.

Sheaf-theoretic methods. Abramsky et al. [16] use sheaves to model contextuality in quantum mechanics; our application to proof search is, to our knowledge, the first use of Grothendieck topologies for tactic design. The judgment geometry series [5, 6, 7, 8] provides the mathematical foundation on which semantic moves operate.

Compositional verification. Compositional program verification has a long history, from Hoare logic and rely-guarantee reasoning to separation logic and concurrent abstract predicates. These frameworks compose proofs across program boundaries but do so within a fixed proof calculus. Semantic moves extend this tradition by introducing a *topological* composition mechanism—the covering-family structure of the semantic site—that captures not just sequential and parallel composition but also hierarchical nesting and modular overlap.

Proof repair. Recent work on proof repair [17] addresses the problem of updating proofs when definitions change. Our `repair` move addresses a different but related problem: modifying the *site structure* (not the proof term) to enable gluing that was previously obstructed. The two notions of repair are complementary and could be combined in a system that supports both term-level and site-level repair.

10.2 Conclusion

We have introduced semantic moves—typed, composable proof-search operations that act on the full geometric proof state. Of the 13 move kinds, 7 are geometric and have no analogue in Lean 4, Coq, or F*. Evaluation on 10 programs shows that 34 of 110 applied moves (30.91%) are geometric, confirming that nearly a third of proof-search activity in judgment geometry lies beyond the reach of term rewriting.

The adaptive control layer and construction loop provide a principled orchestration mechanism with termination guarantees. Future work includes: (i) extending the evaluation to larger programs with richer site structure, where `transport_along`, `repair`, and `descend` succeed more frequently; (ii) integrating semantic moves as an action space for machine-learning-guided proof search; and (iii) implementing a bridge to Lean 4 4 that exports classical move sequences as Lean 4 tactic scripts while flagging geometric residuals.

Acknowledgments. The author thanks the JuGeo development team for the infrastructure that made the experiments possible, and the anonymous reviewers for helpful suggestions on the formal presentation.

References

[1] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and M. von Raumer, "The Lean 4 theorem prover and programming language," in *Proc. CADE-28*, Springer LNCS, 2021.

[2] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, "The hitchhiker's guide to logical verification," 2023, Chapter 5: Tactics.

[3] P.-M. Pédrot, "Ltac2: Tactical warfare," in *Proc. CoqPL Workshop*, 2019.

[4] G. Martínez, D. Ahman, V. Dumitrescu, N. Giannarakis, C. Hawblitzel, C. Hriţcu, M. Naber, A. Rastogi, and N. Swamy, "Meta-F*: Proof automation with SMT, tactics, and metaprograms," in *Proc. ESOP*, Springer LNCS, 2019.

[5] H. Young, "Semantic sites for program verification: A sheaf-theoretic foundation," Judgment Geometry Paper 1, 2024.

[6] H. Young, "The judgment algebra: Evidence, obligation, and trust," Judgment Geometry Paper 2, 2024.

[7] H. Young, "Descent and obstructions in compositional verification," Judgment Geometry Paper 3, 2024.

[8] H. Young, "Trust algebras for multi-source verification," Judgment Geometry Paper 4, 2024.

[9] J.-O. Kaiser, B. Ziliani, R. Krebbers, Y. Régis-Gianas, and D. Dreyer, "Mtac2: Typed tactics for backward reasoning in Coq," in *Proc. ICFP*, ACM, 2018.

[10] D. Matichuk, T. Murray, and M. Wenzel, "Eisbach: A proof method language for Isabelle," *Journal of Automated Reasoning*, vol. 56, no. 3, pp. 261–282, 2016.

[11] J. C. Blanchette, C. Kaliszyk, L. C. Paulson, and J. Urban, "Hammering towards QED," *Journal of Formalized Reasoning*, vol. 9, no. 1, pp. 101–148, 2016.

[12] S. Polu and I. Sutskever, "Generative language modeling for automated theorem proving," *arXiv:2009.03393*, 2020.

[13] K. Yang, A. M. Swope, A. Gu, R. Chalapathy, P. Song, S. Bai, B. Peng, T. Berg-Kirkpatrick, and H. Su, "LeanDojo: Theorem proving with retrieval-augmented language models," in *Proc. NeurIPS*, 2023.

[14] S. Mac Lane and I. Moerdijk, *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*, Springer, 1994.

[15] M. Artin, A. Grothendieck, and J.-L. Verdier, *Théorie des topos et cohomologie étale des schémas (SGA 4)*, Springer LNM 269/270/305, 1972–1973.

[16] S. Abramsky and A. Brandenburger, "The sheaf-theoretic structure of non-locality and contextuality," *New Journal of Physics*, vol. 13, 113036, 2011.

[17] T. Ringer, R. Just, T. Limpanichkul, and D. Grossman, "Proof repair across type equivalences," in *Proc. PLDI*, ACM, 2021.